

Sparse Goal Edits and Variable-Duration Options in World-Model Hierarchical RL

George Vasilache

Working draft — July 13, 2026

Abstract

We study two restrictions on the manager of a Director-style hierarchical agent built on DreamerV3: (i) *masked goal edits*, where the manager maintains a persistent goal code and edits only a sparse subset of its blocks per decision, and (ii) *variable decision durations*, where the manager chooses how long each goal is held. Both are motivated by decision sparsity: fewer and more local manager interventions per environment step. On dense-reward cartpole the combined recipe outperforms each component alone and reaches returns of ~ 840 , competitive with the best fixed-schedule variants; on cheetah, replacing fixed sparsity/duration targets with Lagrangian controllers roughly doubles the return. On sparse-reward tasks (hopper hop, acrobot swingup) every variant tested so far fails to encounter reward, while the unrestricted Director baseline learns. A controlled comparison isolates the variable-duration mechanism at short target durations as sufficient for this failure, and ongoing experiments test whether duration targets, duration–return coupling, or goal-edit locality is responsible.

1 Setting

The base agent is a reimplementation of Director-style hierarchy on DreamerV3. A recurrent state-space world model provides latent states s_t . A goal autoencoder discretizes latent goals: an encoder maps a target state to a code $z \in \{1, \dots, C\}^L$ of L categorical blocks ($L=8$ on the tasks used here), and a decoder maps codes back to latent goals. A *manager* policy selects codes every K steps (Director: fixed $K=8$) and is trained on extrinsic plus exploration reward; a *worker* policy receives the decoded goal and is trained on a goal-reaching reward. All training happens in imagination; the worker never observes task reward directly.

We investigate whether manager decisions can be made *sparser* without loss of control performance, along two axes:

Masked goal edits. The manager maintains a running goal code Z_t and at each decision outputs a new code z_t together with a binary mask $m_t \in \{0, 1\}^L$, updating

$$Z_t = m_t \odot z_t + (1 - m_t) \odot Z_{t-1}, \quad (1)$$

so only the selected blocks change and the rest of the goal persists.

Variable durations. Alongside the goal, the manager outputs a hold duration $d_t \in \{1, \dots, 16\}$; the next decision occurs after d_t steps rather than on a fixed schedule.

The natural cost measure combining both axes is the average number of goal blocks edited per environment step,

$$\text{blk/step} = \frac{\mathbb{E}[\|m\|_1]}{\mathbb{E}[d]} = \frac{\bar{m} L}{\bar{K}}, \quad (2)$$

where $\bar{m}$ is the mean edit fraction and $\bar{K}$ the realized mean duration. Director corresponds to $\bar{m}=1$, $\bar{K}=8$, i.e. $\text{blk/step} = 1$.

Motivation. If performance can be held while blk/step drops well below 1, manager decisions become temporally extended *and* spatially local: the goal stream turns into a piecewise-constant program with sparse diffs. Anticipated benefits are (a) interpretability—individual block edits can be attributed to behavior changes; (b) a controllable knob for temporal abstraction; (c) less non-stationarity in the worker’s objective, since most of the goal persists across decisions.

2 Sparsity and duration control

Direct penalties on edit fraction or duration proved brittle (Sec. 4); the current recipe controls both with dual-ascent Lagrangian controllers of the same form. For a statistic x (e.g. mean mask probability) with target τ , a multiplier λ is updated toward satisfying the constraint and the term λx is added to the manager loss:

$$\lambda \leftarrow \text{clip}(\lambda \cdot \exp(\eta(x - \tau)), \lambda_{\min}, \lambda_{\max}). \quad (3)$$

Three such controllers are active in the combined (“group D”) recipe:

- **Edit-rate:** mean mask probability $\rightarrow$ target 0.3 (mode **prob**).
- **Per-block entropy:** mean per-block mask entropy $\rightarrow$ target 0.5, preventing collapse onto always/never-edited blocks (mode **entropy**; **prob_entropy** runs both).
- **Duration:** the switch-weighted deviation $|\mathbb{E}[d] - \tau_d| \rightarrow$ tolerance 0.1, with $\tau_d \in \{4, 8\}$. Regulating the deviation (rather than raw duration) keeps the controller symmetric; it relaxes to $\lambda \rightarrow 0$ once within tolerance.

Additionally, a *structural consistency* loss with weight w_s (default 200) ties code-space distances to latent state distances, so that small code edits correspond to nearby goals:

$$\mathcal{L}_{\text{struct}} = \|\text{dist}(z_i, z_j) - \text{dist}(s_i, s_j)\|^2, \quad \text{realized corr.} \approx 0.92\text{--}0.98. \quad (4)$$

The manager’s per-decision reward aggregates the extrinsic reward over the held segment with a *mean* by default, $r_i^{\text{mgr}} = \frac{1}{d_i} \sum_{t \in \text{seg}_i} r_t$, which makes the manager’s return invariant to how finely time is partitioned; a *sum* aggregation ($r_i^{\text{mgr}} = \sum_{t \in \text{seg}_i} r_t$, Director-style) couples return to duration and is under test (Sec. 5).

3 Experimental setup

Tasks are DeepMind Control from pixels (+proprio): cartpole swingup and cheetah run (dense reward), hopper hop and acrobot swingup (sparse/impulsive reward). Two scales are used (Table 1); all runs are 4M environment steps, seed 0, on the Saion cluster. Experiments are numbered globally (**en**) and logged in `EXPERIMENTS.md`; scores below are means over the last 10 episodes (“last10”) and the best trailing-10 window (“peak”).

Scale	Image	Model	Batch	Hardware
small	32 ²	size6m	16 × 32	1×V100
BIG (<code>director_match</code>)	64 ²	deter 1024, 512-wide MLPs	16 × 64	1×A100-80GB

Table 1: Run scales. BIG matches the published Director architecture.

4 Results to date

Task	Recipe (BIG scale)	last10	peak	Reference
cartpole	combined mask+var- K +struct (D)	776	844	e171 (e157: 833/869)
cartpole	mask only, fixed $K=8$	360	419	e163 (history: e9 $\approx$ 1)
cartpole	var- K only, target 4	97	658	e167 (collapsed)
cartpole	plain var- K + struct, target 8	725	758	e46 (small scale)
cheetah	combined, Lagrangian controllers	465	508	e175
cheetah	combined, fixed targets	253	–	e142
hopper	any masked / var- K variant	0	$\sim$ 6	e164–e173 (all)
hopper	Director baseline, fixed $K=8$	$\sim$ 300	–	e124
acrobot	combined (D), targets 4	$\sim$ 1	37	e176/e177

Table 2: Key results. Returns are episode scores (max 1000).

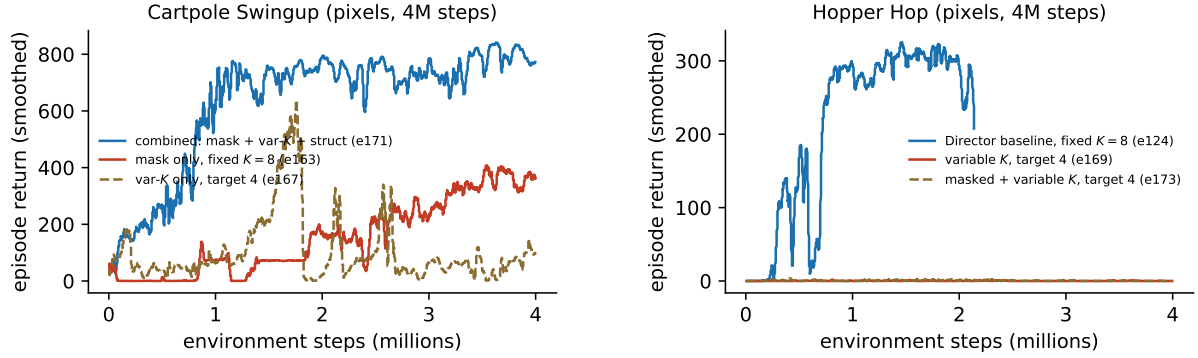


Figure 1: Left: on cartpole the combined recipe (mask + variable K + struct) clearly outperforms either component alone. Right: on hopper the Director baseline (fixed $K=8$, full goal replacement) learns, while the variable-duration variant with target 4 — differing *only* in the variable-duration package — never encounters reward; adding masking does not change this.

1. The components interact; neither suffices alone. On cartpole, masked edits at fixed $K=8$ and variable K without masking/struct both underperform the combined recipe by a large margin (Table 2, Fig. 1 left). This is consistent across the program’s history: masked fixed- $K=8$ variants collapsed or plateaued (best 511), and plain variable- K needed both a duration prior and the structural loss to reach 724. The combined recipe achieves blk/step ≈ 0.6 – 1.1 at returns of 776–844, versus Director’s blk/step = 1.

2. Lagrangian controllers transfer better than fixed penalties. Replacing fixed sparsity/duration penalty weights with the controllers of Eq. (3) left cartpole unchanged but roughly doubled cheetah (253 $\rightarrow$ 465–508), the best cheetah result in the program. The duration controller behaves as intended: the multiplier relaxes to 0 once $|\mathbb{E}[d] - \tau_d|$ is within tolerance, and the realized duration stays at the target without an active constraint.

3. Sparse-reward tasks fail under every restricted variant. On hopper, every masked and/or variable-duration configuration tested (16+ runs across recipes, scales, and controller variants) shows manager extrinsic reward $\approx 10^{-4}$ throughout training — the reward is never encountered, so nothing downstream can learn. The Director baseline (e124) finds reward within 0.2–0.4M steps and reaches ~ 300 . Acrobot behaves like hopper under the combined recipe.

4. A controlled comparison implicates short variable durations. Resolved-configuration diffs show e169 (variable K , duration target 4, no mask, no struct) and e124 (fixed $K=8$) differ *only* in the variable-duration package (Fig. 1 right). Two mechanisms could explain the failure: (i) *commitment* — re-deciding every ~ 4 steps may not persist long enough with one goal to leave the low-reward region; (ii) *duration–return decoupling* — with mean aggregation (Sec. 2) the manager has no return incentive to hold longer. Goal-edit *locality* (mask + struct + persistent goal restricting how far the proposed goal can move per decision) remains a candidate for the masked variants, which fail on hopper even at fixed $K=8$ with high worker action entropy.

5 Ongoing experiments

Seven runs launched 2026-07-13 (all BIG, 4M steps) separate these hypotheses. Early signals at ~ 0.12 M steps are annotated; the “alive” criterion is reward rate $> 10^{-2}$ by 1M steps (e124 reference).

If e178 learns hopper, short duration targets — a cartpole-tuned setting carried into every variable- K run — are the primary failure cause, and the fix is per-task (or adaptive, lower-bounded) duration targets. If e178 stays flat but e181 learns, the mean aggregation is the cause. If both stay flat while e180 learns acrobot, attention shifts to the variable-duration machinery itself and to goal-proposal locality; the next interventions would be mixing a small fraction of task reward into the worker and allowing periodic non-local goal proposals.

Exp	Configuration	Task	Tests	Early signal
e178	var- K , target 8 (e169 + one knob)	hopper	commitment (i)	reward rate 1.5×10^{-3}
e179	var- K , target 8	cartpole	does struct matter at $\tau_d=8$?	learning (141 @ 0.12M)
e180	Director baseline	acrobot	is acrobot hard for the base?	<i>learning</i> (0.13 rate)
e181	e178 + sum aggregation	hopper	decoupling (ii)	0.9×10^{-3}
e182	combined D, target 8	hopper	rescue of full recipe	flat
e183	combined D, $10 \times$ expl. weight	hopper	exploration strength	flat
e184	combined D, target 8	acrobot	rescue of full recipe	marginal

Table 3: Hypothesis matrix. A separate pair (e160/e161) ablates the structural loss ($w_s=0$) within the combined recipe on cartpole/hopper.

6 Outlook

The combined recipe currently matches strong fixed-schedule baselines on dense tasks at comparable or lower edit rates, and the Lagrangian controllers remove per-task penalty tuning. The open problem is sparse-reward tasks. Any claim of benefit for sparse manager decisions must include a task where reward has to be *found*; the hypothesis matrix above is designed to identify the mechanism, after which the recipe will be revised and re-evaluated across all four tasks.